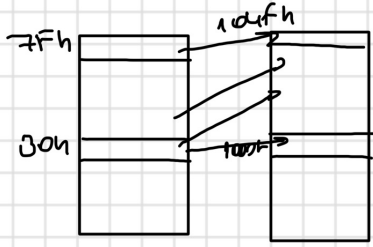


$$5.16 \times 10^9 = 5.16 \times 10^9$$



```
mov A, 30h
mov DPTR, #1000h
MOVX @DPTR, A
mov A, 31h
mov DPTR, #1001h
mov @DPTR, A
```

X

bunlar
yeni
dönüşüm kullanalım.

Kud

```

telrar: MOV    DPTR, #1000h    ; hedef adres
        MOV    R0, #30h        ; Kaynak adres
        MOV    A, @R0
        MOVX   @DPTR, A
        INC    R0
        INC    DPTR
        CJNE   R0, #80h, telrar ; Önce artırıyoruz sonra kontrol ediyoruz
        END                ; bundan dolayı 80h yazdık

```

DSNZ $\Rightarrow$ displacement jump not zero $\neq 0$ despite atla

امتیاز 2

```

mov     DPTR, #1000h
mov     R0, #30h
mov     R7, #50h

telvar:
    mov     A, @R0
    movx    @DPTR, A
    inc     R0
    inc     DPTR
    djnz    R7, telvar
    end

```

başlangıç kalıplarımız

org	0000h
sgmp	başla
org	0030h
başla:	

0-30h arasında kesme sinyali var

milano - keil drucker

Soru:

30h ile 50h adreslerindeki verilerin toplanan programı yazınız.

- a-) Taşma olmadan
b-) Taşma olursa

mov 30H, #01H
 mov 31H, #03H
 mov 32H, #21H
 mov 33H, #02H
 ;
 ;

} 30 to 50
 address
 very expensive
 (memory)

A şikâi:

```
mov RO, #30h
mov A, #00h ; formu 2 kez gâirdifimrda depe kalmaması için
topla: ADD A, @RO ; dizinin 1. elemanı ile A'ya toplama
INC RO
CJNE RO, #51h, topla
mov 60h, A ; toplamı istenen adrese yaz
END
```

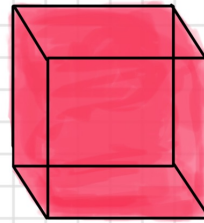
Azama azama istiyorsak MOV 60h, A döngünün içine yazılmalı. her toplam sonucu depe görmek isterseniz

```
mov RO, #30h
mov A, #00h
ADD A, @RO
mov 60h, A
INC RO
CJNE RO, #51h, topla
END
```

b-şikâi taşıma olma durumunda

düşük bit 6LH yüksek bit 60h

```
mov RO, #30h
mov A, #00h
mov 60h, #00h
mov 6LH, #00h
topla: ADD A, @RO
JNC elde_yok
INC 60h ; elde var bilgisi
elde_yok: INC RO
CJNE RO, #51h, topla
mov 6LH, A
END
```



Yeni Soru:

30h ile 50h arasındaki en büyük olanı bulma

```
mov 60h, 30h ; ilk elemanı adrese yazıyoruz
mov RO, #3LH ; dizinin sonraki elemanınıyla devam ediyoruz
tekrar: mov A, @RO ;
CJNE A, 60h, eşit_değil ; CJNE kılavuz yapıyor ama A değeri depelemiyoruz
SJMP dengi_kontrol ; eşit bölge
```

eşit değil:

```
JC yenisayf_kucuk
mov 60h, A ; yeni sayf büyük bölge
```

yenisayf_kucuk: ; deplasman yapma
dengi_kontrol:

```
INC RO
CJNE RO, #51h, tekrar
```

2. Sık 60 adresine en büyüğüne indisi 61 yazdırılır
 mov b1H, #30h
 mov b0H, 30H ; ilk elemanı adrese yazıyoruz
 mov RO, #31H ; dizinin sonraki elemanımızla devam ediyoruz
 tekrar; mov A, @RO;
 CJNE A, b0H, eşit değil
 SJMP dengukontrol ; eşit değilse

eşit değil:
 JC yenisayı küçük
 mov b0H, A
 mov b1H, RO
 yenisayı küçük: ; değisim yapma
 dengukontrol:
 INC RO
 CJNE RO, #51h, tekrar

3. Sık 4. Eğer 2 tane en büyük varsa

mov b1H, RO

SJMP dengukontrol

Sık 4i

mov b2H, #01
 mov b1H, #30h
 mov b0H, 30H ; ilk elemanı adrese yazıyoruz
 mov RO, #31H ; dizinin sonraki elemanımızla devam ediyoruz
 tekrar; mov A, @RO;
 CJNE A, b0H, eşit değil
 mov b1H, RO ; eşit değilse
 INC b2

SJMP dengukontrol
 eşit değil:
 mov b2H, #01h
 JC yenisayı küçük
 mov b0H, A ; yeni sayı büyük bölge

yenisayı küçük: ; değisim yapma
 dengukontrol:

INC RO
 CJNE RO, #51h, tekrar

Teke Gift Örneği

```
MOV R0, #20H ; dizinin ilk indisi
MOV R1, #40H ; son indisi
MOV B0, #0DH ; arift dizisinin eleman sayısı
MOV B1, #01H ; en büyük teke sayısı
```

```
MOV A, @R0 ; kaynak diziden oku
JB ACC.0, sayitek ; Bit seklinde yazılacaksa bu bit
```

; sayı arift bölge

```
MOV B, #02H
```

```
MOV AB
```

```
MOV @R1, A
```

```
INC R1 ; dizi indisini artır
```

```
INC B0H
```

```
LJMP dogru kontrol
```

sayı tekse:

```
CJNE A, B1H, ariftdegi
```

; son adresi testlersam MOV B2H, R0

```
LJMP dogru kontrol
```

esit degil:

```
JC yesisayi kucuk
```

```
MOV B1H, A
```

```
MOV B2H, R0
```

yesisayi kucuk

STMP dogru kontrol ; mantıksız bir atlama

dogru kontrol;

```
INC R0
```

```
CJNE R0, #3AH, tekrar
```

→ burası 40 değil 3A

END

